

JavaScript

INTRODUCTION

Le JavaScript est un langage de programmation de script. Il permet de manipuler et d'implémenter des fonctionnalités complexes, pour rendre vos pages web plus interactives et dynamiques.

En savoir plus

Ce langage a été inventé par Brendan Eich en 1995, soit un an après le langage CSS. Depuis 2015, avec la sortie d'ECMAScript 2015 (souvent appelé ES6), l'évolution de JavaScript suit un cycle annuel. Plutôt que de parler de JavaScript 7, JavaScript 8, etc., on fait généralement référence aux versions de la norme ECMAScript : ECMAScript 2020, ECMAScript 2024, ECMAScript 2025, etc.

LIER SON FICHIER JAVASCRIPT

Comme pour le fichier HTML et le CSS, le JavaScript se crée depuis un éditeur de texte en l'enregistrant en **.js**

Pour associer un fichier JavaScript à son fichier HTML il faut placer une balise `<script>` qui peut recevoir différents attributs.

Ainsi, on peut spécifier le "chemin" de notre fichier JavaScript par rapport au fichier HTML :

`src="javascript.js"`

```
<script src="script.js"></script>
```

On peut également mettre du JavaScript directement dans le fichier HTML à l'intérieur d'une balise `<script>`.

```
<script>
  /* Ici je peux mettre du JavaScript */
</script>
```

À noter : Les appels aux fichiers JavaScript, se font en fin de fichier, avant la fermeture de la balise `</body>` et non pas dans la balise `<head>` de notre fichier.

Un commentaire JavaScript s'écrit comme en CSS, sous la forme :

```
/* Ceci est un commentaire*/
```

Ou sur une seule ligne :

```
// Un commentaire sur une ligne seulement.
```

LES VARIABLES

Une variable est un identifiant qui permet de stocker une valeur, qui comme son nom l'indique peut varier. On fait appel à une variable pour pouvoir faire référence à cette valeur de manière dynamique ou pour pouvoir la modifier.

Déclarer des variables

Pour déclarer une variable on écrit :

```
let numero
```

On peut directement attribuer une valeur initiale à notre variable lorsqu'on la déclare :

```
let numero = 2;
```

Une constante est un identifiant qui permet de stocker une valeur, mais à l'inverse d'une variable sa valeur va rester constante. Pour déclarer une constante on écrit :

```
const numero = 2;
```

On peut déclarer plusieurs variables ou constantes directement avec une virgule :

```
let variable1 = 1,  
    variable2 = 2;
```

Les noms des variables et des constantes ne peuvent pas contenir d'espace ni de tiret. Pour donner un nom composé on va simplement mettre en majuscule le mot suivant. Par exemple :

```
let monNomDeVariable
```

Il n'est pas rare de trouver des var plutôt que des const ou des let dans des fichiers de script ; var était utilisée dans de nombreux langages et surtout en JavaScript avant la version ES6, et servait à la fois à déclarer des variables et des constantes (un mélange de const et de let donc).

Modifier des variables

Pour modifier une variable on va l'appeler et lui indiquer sa nouvelle valeur :

```
numero = 3;
```

On peut aussi utiliser plusieurs méthodes pour additionner, soustraire, etc. des valeurs ou des variables. Pour cela on utilise des **opérateurs** :

- = Assigne une valeur.
- + Permet d'ajouter une valeur.
- - Permet de soustraire une valeur.
- * Permet de multiplier une valeur.
- / Permet de diviser une valeur.
- += Ajoute et assigne la valeur.
- -= Soustrait et assigne la valeur.
- *= Multiplie et assigne la valeur.
- /= Divise et assigne la valeur.
- ++ Permet d'incrémenter de 1.
- -- Permet de décrémenter de 1.

Exemple :

```
let nombre1 = 2,  
    nombre2 = 6,  
    total = nombre1 + nombre2 ; // La variable totale est égale à 8
```

Autre exemple :

```
let nombre = 2;  
total += 8 ; // Le résultat sera 10
```

Autre exemple :

```
let nombre = 11;
nombre++; // Le résultat sera 12
```

LES TYPES DE DONNÉES

Les trois principaux types de données primitives en JavaScript sont :

- **Les nombres** :

```
let numero = 2; // ou const numero = 2 si c'est une constante;
```

- **Les chaînes de caractères** (du texte) :

```
let texte = "Ceci est une chaîne de caractères"; // ou const texte
```

- **Les valeurs logiques, appelées booléens** (vrai ou faux) :

```
let vrai = true; // const fonctionne aussi pour les booléens si la valeur ne change pas
let faux = false; // ici, on pourrait avoir const isAdmin = false;
```

Il est possible d'appeler différents types de données. Par exemple :

```
let paragraphe = 'paragraphe numero' + numero;
// ou encore comme ceci :
const text = `Le numéro est le suivant : ${numero}`;
```

On peut vérifier nos valeurs dans la « console » de notre navigateur (grâce à l'inspecteur d'élément) en utilisant la fonction « `console.log()` » :

Par exemple :

```
console.log(maVariable);
```

Dans la console on verra alors la valeur de notre variable.

LE DOM

DOM signifie Document Object Model (Modèle d'Objet du Document).

C'est la représentation de l'interface HTML. On va donc pouvoir grâce au JavaScript manipuler notre HTML et notre CSS.

Pour accéder à un élément de notre structure HTML on va devoir utiliser un sélecteur :

- **getElementsByTagName** : sélectionne une balise.
- **getElementById** : sélectionne un attribut `id`.
- **getElementsByClassName** : sélectionne un attribut `class`.
- **querySelector** : permet de sélectionner un sélecteur complexe (avec la même syntaxe qu'en CSS) : on va privilégier l'usage de `querySelector()`.
- **querySelectorAll** : permet de sélectionner plusieurs éléments.

Par exemple, on écrit :

```
const demoContainer = document.querySelector("#demo");
```

« `document` » permet la recherche de l'élément que l'on souhaite sélectionner dans la page.

Autre exemple :

```
const allDemos = document.querySelectorAll("#demo > div");
```

MODIFIER LE DOM

Créer, ajouter et supprimer des éléments

Pour créer un élément on va utiliser la fonction « `createElement` » :

```
const nouvelleDiv = document.createElement("div");
```

Maintenant que notre élément est créé on va pouvoir l'ajouter à notre structure. Pour cela on peut appeler la variable qui sera son élément parent et utiliser la fonction « `appendChild` » :

```
const demoContainer = document.querySelector("#demo");  
const nouvelleDiv = document.createElement("div");  
demoContainer.appendChild(nouvelleDiv);
```

Si on souhaite supprimer un élément, on peut utiliser la fonction « `removeChild` » :

```
demo.removeChild(ancien) ;
```

Insérer ou modifier du contenu

Pour insérer ou modifier le contenu d'un élément on va utiliser la fonction « `innerHTML` ». La valeur sera alors une chaîne de caractères (pouvant contenir du code HTML) ou une variable.

Par exemple :

```
const demoContainer = document.querySelector("#demo");  
demoContainer.innerHTML = "<p>Mon contenu HTML</p>";
```

Si par exemple, notre élément contient un attribut, il y aura alors une confusion entre les fermetures et les ouvertures des guillemets dans la chaîne de caractère. Il faut donc changer le type de guillemet. Par exemple :

```
demo.innerHTML = "<img src='mon-image.png'>";
```

On peut également utiliser la propriété `textContent` pour lire, écrire ou remplacer le texte d'un élément. Contrairement à `innerHTML`, le contenu est traité comme du texte brut : les balises HTML ne sont donc pas interprétées.

```
const nouveauParagraphe = document.createElement("p");  
nouveauParagraphe.textContent = "Je suis un nouveau paragraphe";
```

Créer et modifier des attributs

Pour ajouter ou modifier un attribut et sa valeur, on va utiliser la fonction « `setAttribute` ». Il va donc falloir lui donner deux paramètres :

```
const demoContainer = document.querySelector("#demo");  
  
demoContainer.setAttribute("href", "mon-lien.html");
```

Il est aussi possible de donner le nom de l'attribut directement et de lui assigner une valeur :

```
demoContainer.href = "mon-lien.html";
```

Si on souhaite récupérer la valeur d'un attribut on va utiliser la fonction « `getAttribute` ».

```
demo.getAttribute("href");
```

Si on souhaite supprimer un attribut on va utiliser la fonction « `removeAttribute` ». On aura alors besoin de lui donner uniquement le nom de l'attribut à retirer.

```
demo.removeAttribute("href");
```

Ajouter, supprimer des classes

Pour accéder à la liste des classes d'une balise, on va utiliser la fonction « `classList` ».

On peut ajouter une classe :

```
demo.classList.add("nouvelleClasse");
```

On peut aussi supprimer une classe :

```
demo.classList.remove("ancienneClasse");
```

Changer le style

Pour changer le style CSS d'un élément, on va utiliser la fonction « `style` ». Il faut donc cibler la variable dont on souhaite changer le style puis, faire appel à la fonction, nommer la propriété CSS que l'on veut modifier. Enfin on va lui passer le nouveau paramètre, par exemple :

```
demo.style.color = "red";
```

Pour rappel, en JavaScript, les noms de variables, de fonction, etc. ne peuvent pas avoir d'espace ni de tiret. Une propriété CSS composée aura donc la forme :

```
demo.style.backgroundColor = "red";
```

ÉCOUTER DES ÉVÉNEMENTS

Un événement est une réaction, liée à une action faite par l'utilisateur. On l'utilise grâce à la méthode « `addEventListener` ». Par exemple si on souhaite changer le style d'un élément au clique sur un élément :

```
const demoBtn = document.querySelector("#demo-button");
const demoImg = document.querySelector("#demo-image");

demoBtn.addEventListener("click", function() {
    demoImg.style.display = "none"; // l'image va disparaître lorsqu'on clique sur le bouton
});
```

Il existe de nombreuses actions que l'utilisateur peut faire et sur lesquelles on peut déclencher des événements.

LES FONCTIONS

Une fonction est un groupement d'instructions qui exécutent des tâches. On les utilise pour des mécaniques et des fonctionnalités qui vont être réutilisées, ou pour les appeler à un moment précis.

Pour donner des instructions à une fonction on va devoir la nommer, on écrit alors :

```
function maFonction(){
    // Ici on place les instructions que la fonction va effectuer.
};
```

Ensuite, pour déclencher la fonction on écrit simplement :

```
maFonction();
```

On peut aussi déclencher par exemple une fonction sur un écouteur d'événement :

```
demo.addEventListener("click", maFonction);
```

Pour vérifier si notre fonction se déclenche bien, on peut utiliser une alerte, qui nous ouvrira un message dans notre navigateur :

```
function maFonction(){
    alert("Ma fonction se déclenche !");
};
```

À noter : En JavaScript, les variables créées peuvent être vues ou utilisées qu'à l'intérieur du bloc de code dans lequel elles sont déclarées. Par exemple, si elles sont déclarées dans une fonction, elles n'existeront pas en dehors de celle-ci.